

Report: ffjavascript/snarkjs Service Worker Compatibility in Wallet-Extension

1. Are Your Concerns Right?

Yes, your concerns are valid. The ZK proof generation will face issues in the wallet-extension's MV3 service worker environment.

The Problem

The SDK's `ProofGenerator.ts` (`sdk/src/encryption/zk/identity/ProofGenerator.ts`) uses `@cryptkeeperzk/snarkjs` which depends on `@cryptkeeperzk/ffjavascript`. These libraries have two critical dependencies that are **incompatible with Chrome MV3 service workers**:

1. `URL.createObjectURL()` - Used by `ffjavascript` to create blob URLs for inline workers
2. **Web Workers** - Used for parallelized cryptographic operations

Chrome MV3 service workers cannot: - Spawn nested Web Workers (browser limitation) - Use `URL.createObjectURL()` (no DOM access)

Reference: <https://github.com/iden3/ffjavascript/issues/132>

Current Polyfill Status

The wallet-extension already has polyfills in `wxt.config.ts` (lines 8-71): - `URL.createObjectURL` stub - returns empty data URL - `Worker` stub - returns error on `postMessage`

Problem: These are **stub polyfills** that prevent crashes at import-time but **won't actually work** for ZK proof generation. The `Worker` stub returns an error, so any `snarkjs` operation requiring workers will fail.

2. Possible Solutions

Solution 1: Use `singleThread: true` Option (Recommended - Lowest Effort)

`snarkjs` supports single-threaded mode that bypasses Web Workers entirely:

```
// In SDK's ProofGenerator.ts
const { proof, publicSignals } = await snarkjs.groth16.fullProve(
  circuitInputs,
  wasmPath,
  zkeyPath,
  undefined, // logger
```

```
    { singleThread: true } // ADD THIS
  )
```

Pros: - Minimal code change (SDK-side only) - No architectural changes needed - Works in all restricted environments (service workers, Bun, SES)

Cons: - Slower proof generation (no parallelization) - For large circuits, this could be noticeably slower

References: - <https://github.com/iden3/snarkjs> - documents this option for “Bun, browser extensions, SES envs”

Solution 2: Offscreen Documents API

Use Chrome’s Offscreen Documents API to run ZK proof generation in a DOM-enabled context.

Architecture:

```
Service Worker (background.js)
  â‡’ chrome.runtime.sendMessage
Offscreen Document (offscreen.html)
  â‡’ can use Web Workers
snarkjs proof generation (full parallelization)
```

Implementation: 1. Add offscreen permission to manifest 2. Create `offscreen.html` and `offscreen.js` bundled with extension 3. Use `chrome.offscreen.createDocument()` with reasons: ['WORKERS', 'BLOBS'] 4. Communicate via `chrome.runtime` messaging

Pros: - Full Web Worker support (parallel proof generation) - No SDK changes required - Best performance

Cons: - More complex architecture - Only one offscreen document per profile - Must manage lifecycle (create/close)

Reference: <https://developer.chrome.com/docs/extensions/reference/api/offscreen>

Solution 3: Hybrid Approach (Recommended for Production)

Detect environment and use appropriate strategy:

```
// In SDK or wallet-extension wrapper
async function generateProof(...args) {
  if (isServiceWorkerContext()) {
    // Use singleThread for service worker
    return snarkjs.groth16.fullProve(...args,
      { singleThread: true })
  } else {
    // Use full parallelization elsewhere
    return snarkjs.groth16.fullProve(...args)
  }
}
```

```

}

function isServiceWorkerContext() {
    return typeof ServiceWorkerGlobalScope !== 'undefined'
        && self instanceof ServiceWorkerGlobalScope
}

```

Pros: - Works everywhere - Optimal performance where workers are available - Graceful degradation

Cons: - Requires SDK modification

Solution 4: Move ZK Operations to Content Script/Popup

If proof generation is user-initiated (e.g., during a signing flow), run it in a popup or content script context instead of the background service worker.

Pros: - Full DOM/Worker access in popup - No polyfills needed

Cons: - Requires UI to be open during proof generation - Architectural constraint

Solution 5: Fork ffjavascript (Not Recommended)

As mentioned in GitHub Issue #132, some developers forked ffjavascript to disable worker spawning in extension environments.

Cons: - Maintenance overhead (must track upstream changes) - Must also fork snarkjs and any other dependencies - Fragile solution

3. Additional Useful Information

Why @cryptkeeperzk Fork?

The SDK uses @cryptkeeperzk/snarkjs instead of iden3/snarkjs. CryptKeeper (<https://github.com/CryptKeeperZK/crypt-keeper-extension>) is itself a Chrome extension for ZK identity management, so their fork may already have some optimizations for extension environments. Worth investigating their implementation.

Memory Considerations

For web environments, snarkjs recommends minimizing memory allocation:

```
await wtnsCalculate(input, wasmFile, wtns, { memorySize: 0 })
```

Current Polyfill Code Analysis

The existing polyfills in wxt.config.ts: - URL.createObjectURL returns 'data:application/javascript;base64,' (empty) - Worker stub posts error message back immediately

This prevents **import-time crashes** but will cause **runtime failures** when actual ZK operations are attempted.

CSP Considerations

The manifest already has 'wasm-unsafe-eval' which is needed for WebAssembly:

```
"content_security_policy": {  
  "extension_pages": "script-src 'self' 'wasm-unsafe-eval';  
    object-src 'self'"  
}
```

Recommendation

For immediate integration: Use **Solution 1** (singleThread: true) - it's a one-line SDK change that will make ZK work in the service worker context.

For production optimization: Consider **Solution 2** (Offscreen Documents) if proof generation performance becomes a bottleneck. The offscreen document can leverage full parallelization while the service worker coordinates.

For the SDK: Implement **Solution 3** (Hybrid) to make the SDK automatically work across all environments without requiring consuming projects to configure anything.